

SimpleX Chat

Анализ интеграции в маркетплейс
«Уездный кондитер»

Версия документа

1.0 · 2026-07-25

Объект анализа

SimpleX Chat v6.5.6 (stable)

github.com/simplex-chat/simplex-chat

Цель

Оценка пригодности SimpleX как замены
Socket.IO для чата маркетплейса

Рекомендация

**Дополнительный канал, не замена основному
Matrix (Synapse) — лучшая альтернатива**

1. Резюме и рекомендация

SimpleX Chat — это децентрализованный E2E-мессенджер с уникальной архитектурой: в отличие от Signal, Matrix и Telegram, он не назначает пользователям никаких идентификаторов — даже случайных. Вместо центральных аккаунтов используются парные однонаправленные очереди сообщений на релейных SMP-серверах, которые можно развернуть в собственном Docker-контейнере. Аудиты Trail of Bits (2022 и 2024), инвестиции Jack Dorsey (август 2024) и активная разработка (v6.5.6 stable в июне 2025, v7.0.0-beta в июле 2025) делают проект технологически зрелым.

Для маркетплейса «Уездный кондитер» вывод неоднозначный. SimpleX обеспечивает лучший в классе уровень приватности — E2E-шифрование Double Ratchet плюс пост-квантовый алгоритм CRYSTALS-Kyber, отсутствие центральных идентификаторов и поддержка self-hosting всех серверных компонентов. Однако принципиальная невозможность модерации переписки (E2E = админ не видит содержимое) делает SimpleX непригодным для замены текущего Socket.IO-чата, через который проходит основной поток сделок, споры по эскроу и общение с поддержкой.

Итоговая оценка

Критерий	SimpleX	Matrix/Element	Socket.IO (текущий)
E2E-шифрование	✓ Double Ratchet + PQ	✓ Megolm+Olm	☒ нет
Модерация переписки	☒ невозможна	✓ опционально	✓ полная
Self-hosting	✓ Docker, 1 ГБ RAM	✓ тяжелее, 2-4 ГБ	✓ ваш backend
REST API	☒ только WebSocket	✓ REST + SDK	✓ родной протокол
Web-виджет для встраивания	☒ нет	✓ matrix-react-sdk	✓ ваш frontend
Лицензия	AGPLv3	Apache 2.0/AGPLv3	MIT
Совместимость с эскроу-спорами	☒ нет	✓ есть	✓ есть

Рекомендация

SimpleX не следует рассматривать как замену Socket.IO. Однако он отлично вписывается как дополнительный приватный канал поддержки для премиум-клиентов и B2B-сегмента, где конфиденциальность переговоров критична (корпоративные заказы на торты, оптовые поставки ингредиентов, оформление франшизы). В этом сценарии SimpleX работает параллельно с основным чатом — пользователь сам выбирает канал общения. Для полноценной замены Socket.IO с добавлением E2E и сохранением модерации Matrix (Synapse) с matrix-react-sdk технически подходит лучше, но требует существенно больших ресурсов (PostgreSQL, 2-4 ГБ RAM) и более сложной эксплуатации.

2. Архитектура SimpleX Chat

2.1. Принцип «без идентификаторов»

SimpleX — единственный мессенджер без каких-либо user ID. Вместо центральных аккаунтов используется парная адресация per-queue: каждое соединение между пользователями А и В создаёт две однонаправленные очереди (А→В и В→А) на SMP-сервере. В каждой очереди есть два адреса — для отправителя и получателя, плюс опционально третий адрес для push-уведомлений. Идентификаторы очередей генерируются сервером при создании и локальны для этого сервера.

Результат: даже если один и тот же профиль А связывается с В и с С, между этими каналами нет общих метаданных — В и С не смогут подтвердить, что общаются с одним и тем же человеком. Для n пользователей возможно до $n \cdot (n-1)$ очередей, что делает наблюдение графа связей на прикладном уровне вычислительно сложным. Это принципиально отличает SimpleX от Signal (где номер телефона = идентификатор) и Matrix (где Matrix ID @user:server виден всем участникам комнаты).

2.2. Протокол SMP (Simplex Messaging Protocol)

SMP — клиент-серверный протокол передачи сообщений через релейные серверы. Спецификация открыта и опубликована. Протокол вдохновлён Redis serialization protocol, но проще: 10 клиентских команд (NEW, SUB, SEND, GET, ACK, OFF, DEL, SUSPEND, KEY, NID) и 8 серверных ответов. Передача идёт через TLS 1.2/1.3 с ALPN-переговорами (smp/1, xftp/1, http/1.1). Дополнительный слой блочного шифрования TSbChainKeys обеспечивает forward secrecy с блоком 16384 байта и паддингом против traffic analysis. Поддерживается tls-unique channel binding — каждая команда подписана ephemeral ключом очереди против replay-атак.

Критически важно для архитектуры: сервер хранит очередь и сообщения в памяти (по умолчанию) или в опциональном journal-логе. После доставки сообщения адресату оно удаляется с сервера. Это означает, что SimpleX-сервер не может служить архивом переписки — если пользователь потерял устройство и не сделал backup, история теряется безвозвратно. Для маркетплейса, где переписка может быть доказательством в споре по эскроу, это серьёзный риск.

2.3. Типы серверов: SMP, XFTP, NTF

Архитектура SimpleXMQ использует три независимых типа серверов, каждый из которых можно self-хостить отдельно:

SMP server	Передача сообщений через simplex queues. Порт 5223 (TCP) с fallback на 443. Docker-образ simplexchat/smp-server. Реализация на Haskell, in-memory + опциональный journal-лог.
XFTP server	Передача больших файлов (до 1 ГБ) chunked и E2E-шифрованных. Порт 443 (HTTP/2). Образ simplexchat/xftp-server. Файлы хранятся на диске.

NTF server	Push-уведомления для iOS через APNs (iOS не разрешает фоновые сервисы). Образ управляется SimpleX, self-hosting с v6+. Уведомления E2E-шифрованы между NTF и устройством.
WebRTC STUN/TURN	Установление P2P audio/video звонков. Сигналинг через SMP-сообщения, медиа-поток напрямую между пирами.

Для маркетплейса минимальная конфигурация self-hosting — один SMP-сервер (сообщения) + один XFTP-сервер (файлы: фото торгов, сертификаты, документы). NTF-сервер нужен только если в аудитории есть пользователи iOS — для Android push работает через фоновый polling без Google Play Services.

2.4. Возможности чата и E2E-шифрование

SimpleX использует Double Ratchet (как Signal) для E2E-шифрования с добавлением пост-квантового алгоритма CRYSTALS-Kyber с версии 5.6. Это означает защиту даже от будущего квантового компьютера, который может расшифровать записанный сегодня трафик (harvest now, decrypt later). Возможности чата включают: группы до примерно 1000 участников (large groups в активной разработке), файлы до 1 ГБ через XFTP, voice/video звонки 1-на-1 через WebRTC (групповые звонки пока не поддерживаются), read receipts, typing indicators, edit/delete сообщений, reactions, disappearing messages с настраиваемым TTL.

Особо стоит отметить business-адреса, добавленные в v6.2 (сентябрь 2024) — это отдельный тип контакта, при котором каждый клиент подключается к бизнесу через уникальный чат, но бизнес видит все обращения в едином интерфейсе. Это прямое соответствие сценарию «поддержка маркетплейса»: один профиль «Уездный кондитер», к которому клиенты подключаются через QR-код или #simplex-ссылку, а операторы поддержки отвечают из единого CLI или web-клиента.

3. API и варианты интеграции

3.1. Архитектура интеграции: Bot API через WebSocket

У SimpleX нет REST/JSON-RPC API на стороне сервера. Вместо этого предоставляется Bot API — возможность подключиться к запущенному CLI-процессу simplex-chat через WebSocket и обмениваться командами в JSON-формате. Полный справочник команд занимает 43 КБ (2167 строк) и включает создание/удаление контактов, отправку сообщений, управление группами, приём файлов, настройку профилей. Этот подход называется «bot-мост»: backend маркетплейса запускает simplex-chat CLI в headless-режиме, подключается к нему по WebSocket и пересылает события во внутреннюю систему через HTTP webhook.

Архитектурно это означает, что в стеке маркетплейса появляется новый сервис — simplex-bridge. Он работает параллельно с web, chat-service (Socket.IO), db, redis, n8n и caddy. На входе simplex-bridge слушает WebSocket от simplex-chat CLI, на выходе отправляет HTTP-запросы в основной backend маркетплейса. Команды от backend к SimpleX идут в обратном направлении: HTTP от backend к simplex-bridge, затем WebSocket от bridge к CLI, затем SMP-протокол к SMP-серверу.

Пример bot-моста (TypeScript)

```
import WebSocket from "ws"; import axios from "axios"; const SIMPLEX_WS =
"ws://localhost:5225"; // simplex-chat CLI const BACKEND_WEBHOOK =
"https://api.uyezdny.ru/simplex/incoming"; const ws = new WebSocket(SIMPLEX_WS);
ws.on("open", () => { console.log("[bridge] Connected to SimpleX CLI");
ws.send(JSON.stringify({ corrId: "init-1", cmd: "/_users" })); }); ws.on("message", async
(raw) => { const msg = JSON.parse(raw.toString()); if (msg.corrId) return; // response,
skip const resp = msg.resp; if (resp?.type === "newChatItems") { for (const item of
resp.chatItems) { const text = item.chatInfo?.text; const fromContact =
item.chatInfo?.chatInfo?.fromContact?.displayName; await axios.post(BACKEND_WEBHOOK, {
from: fromContact, text, chatItemId: item.chatInfo?.chatItemId, timestamp:
item.chatInfo?.meta?.itemTs, }); } } }); export async function sendToContact(contactName:
string, text: string) { ws.send(JSON.stringify({ corrId: `out-${Date.now()}`, cmd:
`@${contactName} ${text}`, })); }
```

3.2. SDK и клиенты

Официальные SDK: TypeScript (пакет @simplex-chat/types, но в статусе deprecated — рекомендуется прямой WebSocket), Rust (community-порт). Python SDK официально нет, но возможно использовать любую WebSocket-библиотеку (websockets, websocket-client) с ручной сериализацией JSON-команд. Для интеграции в Next.js-приложение «Уездного кондитера» оптимально создать собственный модуль simplex-bridge на TypeScript, который будет работать как edge-сервис.

Критический нюанс: SimpleX не предоставляет web-клиента для встраивания в другие приложения. Пользователь маркетплейса должен установить отдельное приложение SimpleX (iOS, Android, desktop для macOS/Windows/Linux). Это снижает конверсию — далеко не каждый покупатель торта готов устанавливать незнакомый мессенджер ради общения с кондитером. В отличие от этого, matrix-react-sdk позволяет встроить чат непосредственно в web-интерфейс маркетплейса без установки сторонних приложений.

4. Лицензия, compliance, 152-ФЗ

4.1. AGPLv3 — что значит для коммерческого использования

SimpleX Chat распространяется под GNU AGPLv3 — самой сильной copyleft-лицензией. Ключевая особенность AGPL (отличие от GPL) — так называемый Affero clause: если вы модифицируете SimpleX и предоставляете к нему доступ через сеть (SaaS), вы обязаны предоставить исходный код модификаций всем пользователям этого сервиса. Однако это требование распространяется только на модификации самого SimpleX-кода.

Применительно к «Уездному кондитеру» это означает: self-hosting SMP/XFTP-серверов в стандартной конфигурации + использование simplex-chat CLI как чёрного ящика для бота через WebSocket не создаёт AGPL-обязательств для backend-кода маркетплейса. Bot-мост — отдельная программа, общается через протокол, не является derivative work. Но если вы решите форкнуть SimpleX-клиент с другим брендингом или модифицировать simplex-chat CLI с дополнительными функциями — обязаны открыть код модификаций под AGPLv3.

4.2. Совместимость с 152-ФЗ

152-ФЗ «О персональных данных» требует хранение ПДн граждан РФ в РФ, согласие на обработку, возможность удаления и защиту. SimpleX с маркетплейсом в принципе совместим и даёт уникальные преимущества, но не освобождает от регистрации БД ПДн для тех данных, которые маркетплейс собирает о пользователе (имя, телефон, заказы). Self-hosting SMP-сервера в РФ (Selectel, Yandex.Cloud) полностью удовлетворяет требованию о хранении ПДн в РФ. E2E-шифрование — лучший класс защиты переписки. Однако если использовать публичные smr*.simplex.im серверы — это передача данных за рубеж (Нидерланды/Финляндия), что требует отдельного согласия пользователя.

5. Риски для маркетплейса

5.1. Невозможность модерации

Главный риск: E2E-шифрование делает невозможной модерацию переписки. В текущей архитектуре Socket.IO операторы поддержки могут читать переписку, вмешиваться в спор по эскроу, выявлять мошеннические схемы (например, кондитер уговаривает покупателя перевести деньги напрямую в обход платформы). С SimpleX это технически невозможно — даже при судебном запросе сервер не сможет расшифровать переписку, потому что ключи хранятся только на устройствах участников.

Частичное решение: SimpleX с v6.2 поддерживает privacy-preserving content moderation в больших группах — пользователи могут пожаловаться на сообщение, и оно будет расшифровано и показано модератору. Но это работает только для групп, не для direct-чатов, и требует добровольного действия пользователя. Для маркетплейса, где основные споры возникают в direct-переписке «покупатель ↔ кондитер», это не подходит.

5.2. Backup/restore и потеря доступа

SimpleX не имеет централизованного восстановления аккаунта — потому что аккаунта нет. Если пользователь потерял устройство и не сделал backup ключей, он теряет доступ ко всем своим чатам безвозвратно. Для маркетплейса это означает: покупатель, общавшийся с кондитером через SimpleX и потерявший телефон, не сможет продолжить переписку. Создастся новый контакт, история исчезнет. В сценарии эскроу-спора, когда переписка нужна как доказательство, это критично.

Решение: документировать для пользователей обязательность backup. SimpleX позволяет сделать зашифрованный backup-файл и сохранить его в облаке. Но это ответственность пользователя, и в реальности многие её не выполняют.

5.3. Дополнительные риски

- Сложность UX. Пользователь должен установить SimpleX-клиент. Это снижает конверсию — далеко не каждый покупатель торта готов устанавливать незнакомый мессенджер. Matrix с matrix-react-sdk позволяет встроить чат в web-интерфейс маркетплейса.
- Нет voice-сообщений в группах. Group-звонки не поддерживаются — только 1-на-1. Если в будущем потребуется групповая видеоконференция (например, согласование многоярусного торта с дизайнером и кондитером одновременно), SimpleX не подойдёт.
- Зависимость от Haskell-инфраструктуры. Серверы написаны на Haskell. Если потребуется deep-customization (не через конфиг, а через код), нужен Haskell-разработчик — это редкая специализация на рынке РФ.
- Нет официального Python SDK. Для интеграции с backend на Python (если в будущем часть сервисов будет мигрирована) потребуется ручная реализация WebSocket-клиента или использование community-Rust-SDK через FFI.

6. Рекомендуемый сценарий интеграции

6.1. Гибридная модель: Socket.IO + SimpleX

Рекомендуется не заменять Socket.IO, а добавить SimpleX как параллельный канал для премиум-сегмента. В дашборде пользователя появляется выбор канала общения с кондитером: «Стандартный чат» (Socket.IO, модерация, история на сервере) или «Приватный канал SimpleX» (E2E, без модерации, нужен SimpleX-клиент). Кондитер со своей стороны может подключить SimpleX-аккаунт к своему профилю — тогда в его карточке появится QR-код для подключения клиентов напрямую.

Такой подход даёт несколько преимуществ. Во-первых, не нарушается работа основного потока сделок и эскроу — они идут через Socket.IO с полной модерацией. Во-вторых, премиум-клиенты получают реальную приватность для конфиденциальных переговоров (корпоративные заказы, франшиза). В-третьих, кондитеры, работающие с B2B-сегментом, получают профессиональный канал связи без посредничества платформы — это может быть конкурентным преимуществом тарифа PREMIUM.

6.2. Архитектура гибридной модели

В docker-compose.yml добавляются два новых сервиса: smp-server (SimpleX Messaging Protocol server, порт 5223) и simplex-bridge (Node.js-процесс с simplex-chat CLI + bot-мост). SMP-сервер хранит очереди сообщений in-memory с опциональным journal-логом на volume smp_data. SimpleX-bridge подключается к CLI по WebSocket на локальном порту 5225 и пересылает события в основной backend через HTTP webhook на /api/simplex/incoming.

В Prisma-схему добавляется модель SimpleXContact с полями: id, userId (внешний ключ к User), simplexAddress (строка вида smp://fingerprint@host), contactProfileId (внешний ключ к Confectioner или Customer), createdAt. Эта модель связывает внутренний userId маркетплейса с SimpleX-контактом. Когда пользователь генерирует QR-код для подключения, создаётся запись в SimpleXContact; когда клиент подключается, bot-мост получает событие newChatItems и записывает chatItemId в ChatMessage как обычное сообщение типа simplex.

6.3. Этапы реализации

Этап 1 (1-2 дня)	Self-host SMP-сервер в Docker. Конфигурация через переменные окружения. Тестирование с официальными клиентами SimpleX (iOS, desktop). Получение #simplex-адреса.
Этап 2 (3-5 дней)	Создание simplex-bridge сервиса. WebSocket-клиент к simplex-chat CLI. HTTP-webhook в backend. Сохранение входящих сообщений в БД через Prisma. Команды отправки ответов.
Этап 3 (2-3 дня)	Интеграция в UI. Раздел «Приватный канал» в настройках профиля. QR-код для подключения клиентов. Список активных SimpleX-чатов в дашборде.
Этап 4 (2-3 дня)	Документация для пользователей. Инструкция по установке SimpleX-клиента. Backup-стратегия. Уведомление о рисках E2E (нельзя восстановить переписку).
Этап 5 (1-2 дня)	Мониторинг. Healthcheck для SMP-сервера и simplex-bridge. Логирование событий. Alerting при недоступности.

6.4. Альтернатива: Matrix (Synapse)

Если основная цель — не просто добавить приватный канал, а заменить Socket.IO на E2E-чат с сохранением модерации, рассмотрите Matrix (Synapse) + Element. Преимущества: REST Client-Server API, matrix-react-sdk для встраивания web-чата в маркетплейс, опциональное E2E (можно отключить для room, где нужна модерация), Python SDK (matrix-bot-sdk, asyncio), федерация. Недостатки: тяжелее self-host (PostgreSQL обязательно, 2-4 ГБ RAM), более сложная эксплуатация, медленнее чем SimpleX на equivalent-железе.

Для маркетплейса «Уездный кондитер» Matrix технически подходит лучше SimpleX как полная замена Socket.IO — но при этом существенно дороже в эксплуатации. Если цель — минимум усилий с максимальной приватностью, SimpleX как дополнительный канал предпочтительнее. Если цель — архитектурный рефакторинг чата с E2E и модерацией, Matrix + matrix-react-sdk — правильный выбор.

7. Итоговый вывод

SimpleX Chat — технологически впечатляющий проект с уникальной архитектурой без идентификаторов и лучшим в классе E2E-шифрованием. Для маркетплейса «Уездный кондитер» он не подходит как замена Socket.IO, потому что невозможность модерации переписки вступает в противоречие с бизнес-моделью эскроу-платформы, где операторы должны иметь возможность разрешать споры и выявлять мошенничество. Также отсутствие web-клиента для встраивания снижает конверсию — покупатель торта не станет устанавливать отдельный мессенджер.

Однако SimpleX отлично вписывается как дополнительный приватный канал для премиум-клиентов и B2B-сегмента. Гибридная модель «Socket.IO + SimpleX» даёт пользователям выбор: стандартный чат с модерацией или приватный канал с E2E. Это конкурентное преимущество для тарифа PREMIUM и attractive-фича для корпоративных клиентов, заказывающих торты на мероприятия. Реализация займёт 9-15 рабочих дней (5 этапов), добавит в docker-compose два новых сервиса (smp-server, simplex-bridge) и одну модель в Prisma-схему (SimpleXContact).

Если в перспективе планируется полная замена Socket.IO на E2E-чат с сохранением модерации, правильный кандидат — Matrix (Synapse) с matrix-react-sdk. Но это существенно больший проект: миграция существующих чатов, обучение операторов, переработка дашбордов. SimpleX в гибридной модели можно рассматривать как первый шаг к E2E-стратегии маркетплейса — он быстрее выводится в production и не требует миграции основного чата.

Сильные стороны SimpleX для маркетплейса

- Лучший в классе E2E: Double Ratchet + пост-квантовый CRYSTALS-Kyber (защита от harvest now, decrypt later).
- Self-hosting всех серверных компонентов в Docker (SMP, XFTP, NTF) — данные не покидают РФ.
- Business-адреса (v6.2+) — единый профиль поддержки с отдельным чатом на каждого клиента.
- AGPLv3 не затрагивает backend маркетплейса при использовании CLI/сервера как чёрного ящика.
- Активная разработка (v7.0.0-beta в июле 2025), аудиты Trail of Bits, инвестиции Jack Dorsey.

Слабые стороны для маркетплейса

- Невозможность модерации переписки — критично для эскроу-споров.
- Нет web-клиента для встраивания — нужен отдельный SimpleX-клиент у пользователя.
- Backup/restore полностью на пользователе — потеря устройства = потеря истории.
- Нет REST API, только WebSocket к CLI — нестандартная интеграция.
- Групповые звонки не поддерживаются — только 1-на-1.